

✓ PRACTICAL NO 09

Name: Soham B Dikale **Class:** SYA **Roll:** A-27

AIM: Building a Simple Data Science Pipeline: Integrate all learned techniques in previous labs into a complete workflow.

THEORY

1. What is a Data Science Pipeline? Explain its key stages with examples.

Definition

A **Data Science Pipeline** is a structured sequence of steps used to process raw data into meaningful insights and predictions. It helps in organizing the workflow so that tasks like data cleaning, analysis, and modeling are done efficiently and consistently.

Key Stages of a Data Science Pipeline

Stage	Description	Example
1. Data Collection	Gathering data from various sources	CSV files, APIs, web scrapin
2. Data Cleaning	Removing errors and handling missing values	Filling null values
3. Data Preprocessing	Converting data into usable format	Encoding categorical data
4. Exploratory Data Analysis (EDA)	Understanding patterns in data	Graphs, statistics
5. Feature Engineering	Creating new useful features	Extracting age from DOB
6. Model Building	Training machine learning models	Regression, classification
7. Model Evaluation	Checking performance	Accuracy, precision
8. Deployment	Using model in real-world	Web application

Example

For predicting house prices:

- Collect housing data
- Clean missing values
- Encode location
- Scale numerical values
- Train model

- Predict prices

Conclusion

A data science pipeline ensures a **systematic and repeatable workflow**, improving efficiency and accuracy.

2. What is Exploratory Data Analysis (EDA)?

Definition

EDA is the process of analyzing and summarizing datasets using statistical methods and visualizations to understand their structure and patterns.

Purpose of EDA

- Understand data distribution
 - Detect missing values
 - Identify outliers
 - Discover relationships
-

Types of Analysis

A. Univariate Analysis

Analysis of a single variable.

Examples

- Mean, median
- Histogram
- Box plot

Use

- Understand distribution
 - Detect skewness and outliers
-

B. Bivariate Analysis

Analysis of two variables.

Examples

- Scatter plot
- Correlation matrix
- Bar chart

Use

- Identify relationships
- Check dependency between variables

Comparison Table

Feature	Univariate	Bivariate
Variables	One	Two
Purpose	Distribution	Relationship
Example	Histogram	Scatter plot

Conclusion

EDA helps in making **informed decisions before model building**, improving model performance.

3. Differentiate between Label Encoding and One-Hot Encoding

Label Encoding

- Converts categories into numeric values
- Each category gets a unique number

Example:

Category	Encoded
Low	0
Medium	1
High	2

One-Hot Encoding

- Creates separate binary columns for each category

Example:

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0

Comparison Table

Feature	Label Encoding	One-Hot Encoding
Output	Single column	Multiple columns

Feature	Label Encoding	One-Hot Encoding
Order	Yes (may mislead)	No
Memory Usage	Low	High
Best For	Ordinal data	Nominal data

When to Use

- Use **Label Encoding** for ordered data (e.g., small, medium, large)
- Use **One-Hot Encoding** for unordered data (e.g., cities, colors)

Conclusion

Choosing the correct encoding method prevents incorrect assumptions by the model.

4. Why is Feature Scaling Required? Compare Normalization and Standardization

Why Feature Scaling is Required

Feature scaling ensures that all features contribute equally to the model. Without scaling, features with large values dominate smaller ones.

Types of Feature Scaling

A. Normalization (Min-Max Scaling)

Transforms values between 0 and 1:

[$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$]

B. Standardization (Z-score Scaling)

Centers data around mean:

[$X' = \frac{X - \mu}{\sigma}$]

Comparison Table

Feature	Normalization	Standardization
Range	0 to 1	No fixed range
Based on	Min & Max	Mean & Std Dev
Sensitive to Outliers	Yes	Less
Use Case	Neural networks	Most ML models

Conclusion

Feature scaling improves **model performance, speed, and accuracy**.

5. What is Feature Engineering?

Definition

Feature engineering is the process of creating new features from existing data to improve machine learning model performance.

Importance

- Enhances model accuracy
 - Reveals hidden patterns
 - Improves data representation
-

Examples

Original Feature	Engineered Feature
Date of birth	Age
Timestamp	Day, Month
Salary	Salary range
Text data	Word count

Techniques

- Feature creation
 - Feature transformation
 - Feature selection
-

Example

Instead of using:

- Income = 50,000

We create:

- Income Category = Medium

Helps model understand patterns better.

✓ Data Science Pipeline using Iris Dataset

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
df = pd.read_csv("/content/iris.csv")

df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Step 3: Basic Data Understanding (EDA Start)

```
print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())
```

```
(150, 5)
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object  
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000

50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

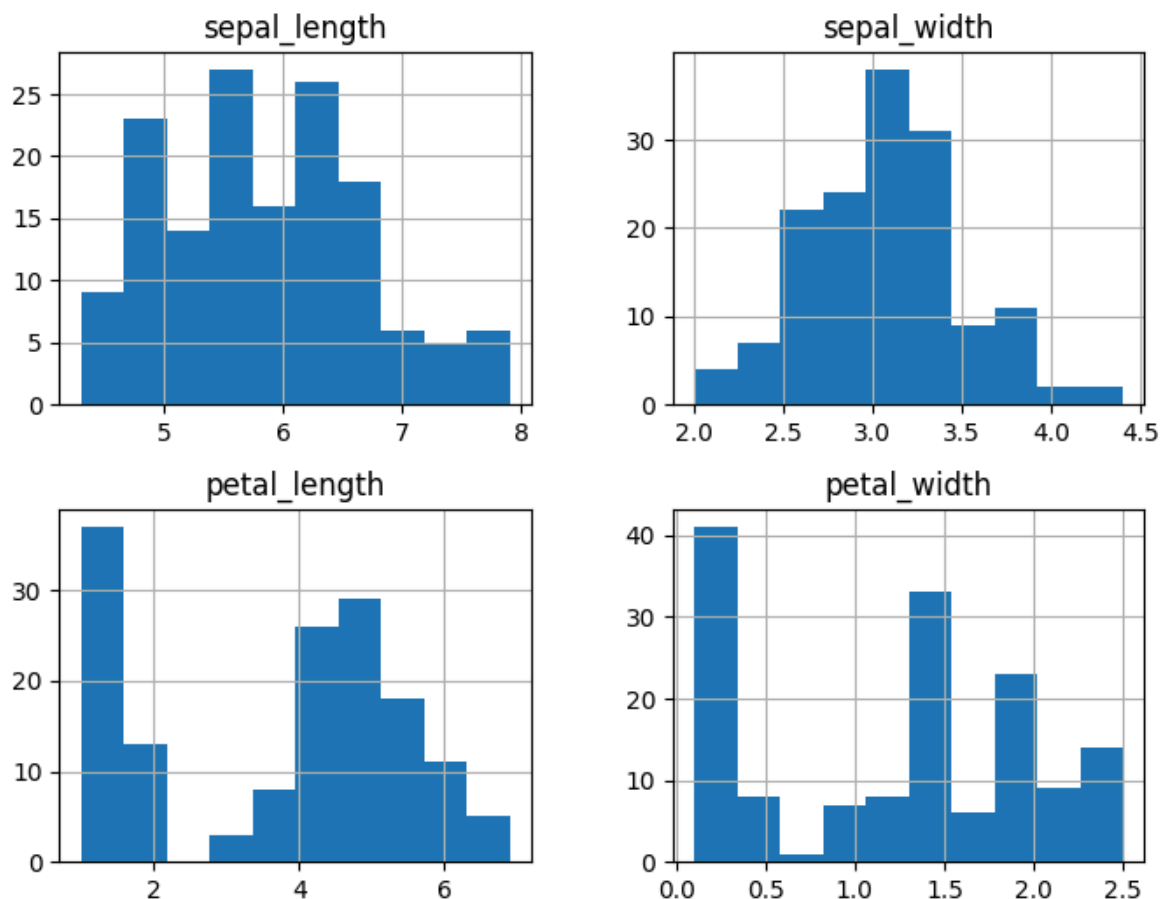
Step 4: Check Missing Values

```
print(df.isnull().sum())
```

```
sepal_length    0
sepal_width     0
petal_length     0
petal_width     0
species         0
dtype: int64
```

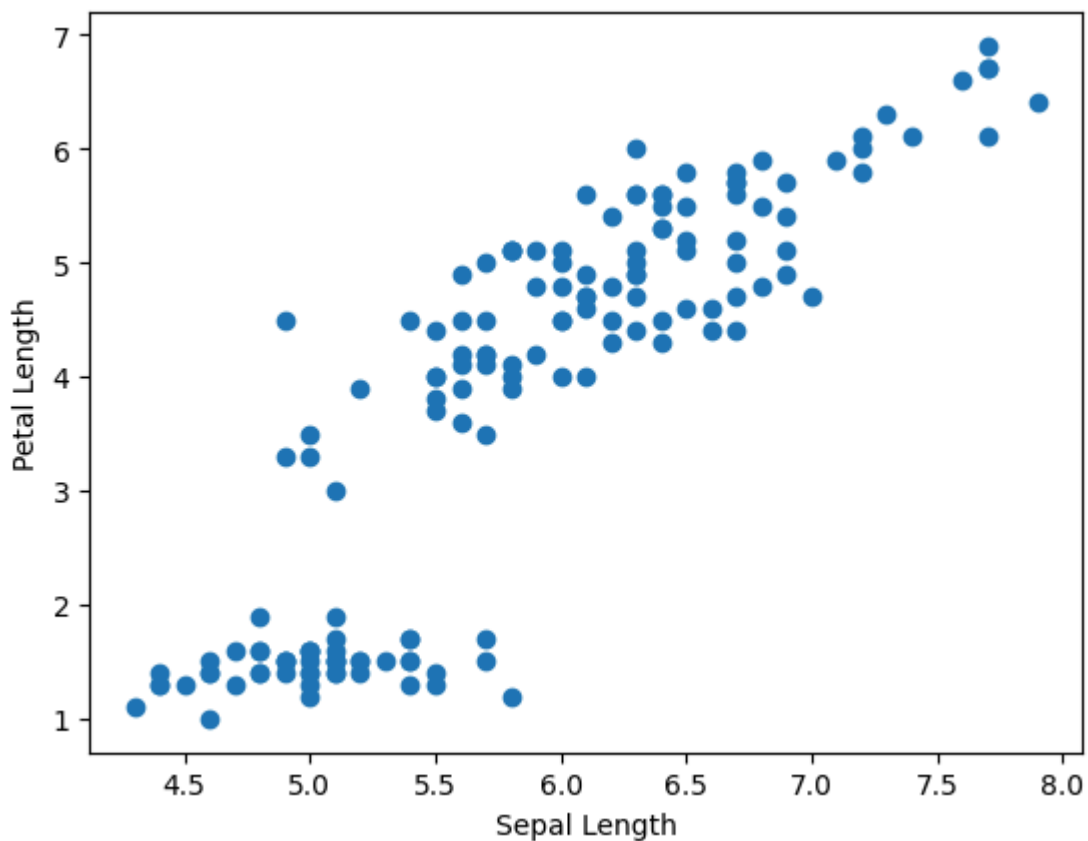
Step 5: Univariate Analysis

```
df.hist(figsize=(8,6))
plt.show()
```



Step 7: Bivariate Analysis

```
plt.scatter(df["sepal_length"], df["petal_length"])
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()
```



Step 8: Feature Engineering

```
df["sepal_ratio"] = df["sepal_length"] / df["sepal_width"]
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_ratio
0	5.1	3.5	1.4	0.2	setosa	1.4571
1	4.9	3.0	1.4	0.2	setosa	1.6333
2	4.7	3.2	1.3	0.2	setosa	1.4688
3	4.6	3.1	1.5	0.2	setosa	1.4839
4	5.0	3.6	1.4	0.2	setosa	1.3889

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 9: Encoding

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
df["species"] = le.fit_transform(df["species"])

df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_
0	5.1	3.5	1.4	0.2	0	1.
1	4.9	3.0	1.4	0.2	0	1.
2	4.7	3.2	1.3	0.2	0	1.
3	4.6	3.1	1.5	0.2	0	1.
4	5.0	3.6	1.4	0.2	0	1.

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 10: Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

features = df.drop("species", axis=1)

scaled_features = scaler.fit_transform(features)

df_scaled = pd.DataFrame(scaled_features, columns=features.columns)

df_scaled.head()
```

	sepal_length	sepal_width	petal_length	petal_width	sepal_ratio
0	-0.900681	1.032057	-1.341272	-1.312977	-1.253256
1	-1.143017	-0.124958	-1.341272	-1.312977	-0.809861
2	-1.385353	0.337848	-1.398138	-1.312977	-1.224046
3	-1.506521	0.106445	-1.284407	-1.312977	-1.185993
4	-1.021849	1.263460	-1.341272	-1.312977	-1.425022

Next steps: [Generate code with df_scaled](#) [New interactive sheet](#)

Step 11: Split Data

```
from sklearn.model_selection import train_test_split

X = df_scaled
y = df["species"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0
```

Step 12: Model Building

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression()
```

Step 13: Prediction

```
y_pred = model.predict(X_test)

print(y_pred)

[1 1 2 2 0 1 0 0 2 2 2 1 2 2 1 1 0 1 1 2 0 2 2 0 0 0 0 0 0 0]
```

Step 14: Model Evaluation

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

Accuracy: 0.9666666666666667
```

✓ Conclusion

The experiment on building a complete data science pipeline using the Iris dataset was successfully performed in Google Colab. The workflow included all essential stages such as data loading, data cleaning, exploratory data analysis (EDA), feature engineering, encoding, feature scaling, model building, and evaluation.

Through this practical, it was observed that each step in the pipeline plays a crucial role in improving the quality of data and the performance of the model. Exploratory Data Analysis helped in understanding the distribution and relationships between variables, while techniques like encoding and scaling ensured that the data was in a suitable format for machine learning algorithms. Feature engineering further enhanced the dataset by creating meaningful new attributes.

The implementation of a machine learning model demonstrated how raw data can be transformed into useful predictions. The evaluation step confirmed the effectiveness of the model.

Overall, this experiment provided a clear understanding of how a structured data science pipeline works in real-world scenarios. It highlighted the importance of proper data preprocessing and systematic workflow in achieving accurate and reliable results in data science projects.

T **B** *I* <> ↺ 🖼️ “ 1/3 ≡ ≡ — Ψ 😊 ☰ [Close](#)

Double-click (or enter) to edit